```
#!/usr/bin/env bash
# harden_v1_2.sh — apply after bootstrap_notices.sh + harden_v1_1.sh. Run from repo root.
# Why: PA accrual runs from breach (or end of the provider relationship), not from the last
# payment. Falling back to last_payment_date can declare a debt "time-barred" too early.
# Now: accrual_date is REQUIRED for the SOL letter. Validates all anchors first; writes nothing on
mismatch.
set -euo pipefail

for f in scripts/generate_notices.py tests/test_generate_notices.py data/sample_debts.json; do
  [ -f "$f" ] || { echo "missing $f" >&2; exit 2; }
done
ts=$(date +%s)
for f in scripts/generate_notices.py tests/test_generate_notices.py data/sample_debts.json; do
  cp -p "$f" "$f.bak.$ts"
done

python3 - <<'PYEOF'
import json
from pathlib import Path

PENDING = []

def edit(path, pairs, replace_all=()):
    s = Path(path).read_text(encoding="utf-8")
    for i, (old, new) in enumerate(pairs, 1):
        n = s.count(old)
        if n != 1:
            raise SystemExit(f"{path}: anchor {i} matched {n}x; nothing written")
        s = s.replace(old, new)
    for old, new in replace_all:
        if old not in s:
            raise SystemExit(f"{path}: bulk anchor {old!r} not found; nothing written")
        s = s.replace(old, new)
    PENDING.append((Path(path), s))

edit("scripts/generate_notices.py", [
    ('''    "account_id", "last_payment_date", "original_creditor", "amount",
    "collector_name",''',
     '''    "account_id", "accrual_date", "original_creditor", "amount",
    "collector_name",'''),
    ('''    basis = parse_date(
        record.get("accrual_date") or record["last_payment_date"], "accrual_date"
    )
''',
     '''    basis = parse_date(record["accrual_date"], "accrual_date")
'''),
    ('''"I understand the four-year limitations period under 42 Pa.C.S. "
            "§ 5525(a), measured from the default or last-payment date on your "
            f"records ({fmt_date(basis)}), ran through''',
     '''"I understand the four-year limitations period under 42 Pa.C.S. "
            "§ 5525(a), measured from the date of default on your "
```

```
            f"records ({fmt_date(basis)}), ran through'''),
    ('''f"    Default / last payment date: {fmt_date(basis)}",'''),
     '''f"    Date of default (accrual): {fmt_date(basis)}",'''),
])

edit("tests/test_generate_notices.py", [
    ('''        "last_payment_date": "2022-05-15",''', '''        "accrual_date": "2022-05-
15",'''),
    ('''    def test_accrual_date_overrides_last_payment(self):
        with self.assertRaises(gn.RecordError):
            gn.generate_pa_sol_notice(sol_record(accrual_date="2023-01-01"), TODAY)
''',
     '''    def test_last_payment_date_is_not_a_substitute_for_accrual(self):
        rec = sol_record(accrual_date=None, last_payment_date="2020-01-01")
        with self.assertRaisesRegex(gn.RecordError, "accrual_date"):
            gn.generate_pa_sol_notice(rec, TODAY)
'''),
], replace_all=[("sol_record(last_payment_date=", "sol_record(accrual_date=")])

sample = Path("data/sample_debts.json")
rows = json.loads(sample.read_text(encoding="utf-8"))
for r in rows:
    if "last_payment_date" in r:
        r["accrual_date"] = r.pop("last_payment_date")
PENDING.append((sample, json.dumps(rows, indent=2, ensure_ascii=False) + "\n"))

for path, text in PENDING:
    path.write_text(text, encoding="utf-8")
print("patched")
PYEOF

export HMAC_SECRET="${HMAC_SECRET:-$(python3 -c 'import secrets;
print(secrets.token_urlsafe(48))')}"
python3 -m unittest discover -s tests
python3 scripts/generate_notices.py --input data/sample_debts.json --dry-run --today 2026-11-05
echo "v1.2 OK. Backups: *.bak.$ts   (records now need accrual_date; last_payment_date is ignored)"
```